

ADVANCED PRACTICE EXAM

Sample Questions

for Claude Architect Exam

Dump 5 · Advanced

66 Questions • Harder distractors • Multi-concept scenarios • Second-order reasoning • Balanced answer keys

Topic Index & Weightage

TOPIC	Q RANGE	COUNT	WEIGHT
 Context & Escalation (Agentic)	Q1-Q12	12	 18%
 Batch API & Extraction	Q13-Q20	8	 12%
 Tool Design	Q21-Q34	14	 21%
 MCP Design Patterns	Q35-Q46	12	 18%
 Agentic Systems & Code Agents	Q47-Q52	6	 12%
 AI Fluency Framework	Q53-Q66	14	 19%

 Higher-weighted topics = more exam questions. Prioritize Tool Design, MCP, and Agentic Systems for maximum score impact.

SECTION



Context & Escalation

ADVANCED — state, failure recovery, and escalation tradeoffs

Q1-Q12

A team's agentic loop has three problems at once: it stops when assistant text contains "done", caps iterations at 10 as the primary stop, and treats any text alongside a tool call as completion. You can fix only one root cause that makes the others moot. Which fix removes the underlying flaw they all share?

A Drive termination solely from `stop_reason` ("tool_use" continues, "end_turn" stops).

✓ CORRECT

B Raise the iteration cap from 10 to 50 so legitimate long tasks complete.

C Expand the completion-phrase list to include "done", "finished", and "complete".

D Only treat text-with-tool-call as completion when the text is longer than 40 words.



Answer A — All three are variants of one anti-pattern: inferring termination from something other than `stop_reason`. Tuning the heuristics (B-D) leaves the root cause intact. All text- and counter-based heuristics.

Between iterations you append a one-sentence natural-language summary of each tool result to history (to save tokens), instead of the structured result. The agent's next-step reasoning degrades. What is the most precise explanation?

A Claude can't read summaries, only JSON.

B Summaries strip the structured fields the model needs to reason about the next action.

✓ CORRECT

C max_tokens is truncating the summary — a tempting shortcut that skips the underlying fix.

D Summarizing breaks statelessness and must never be done.



Answer B — Tool results must be appended so the model can reason on them. Replacing structured results with prose summaries loses the precise fields the next decision depends on; trimming to relevant fields preserves them. The raw (trimmed) result should be appended.

Over a 60-turn dispute, you must both control context growth AND never lose exact figures the customer stated ("847", "by Friday"). Which design satisfies both, and why do the others fail?

A Summarize aggressively; the model can re-derive figures from the summary — a choice that looks simpler but shifts the problem elsewhere.

B Never summarize; pass the full transcript every turn — an approach that treats the symptom rather than the cause.

C Keep a persistent case-facts block (exact amounts/dates/IDs) outside the summarized history, and summarize the rest.

✓ CORRECT

D Store figures only in the system prompt at turn 1 and summarize everything after.

 Answer C — A separate case-facts layer preserves exact values while the conversational remainder is summarized. Aggressive summarization (A) loses precision; full transcript (B) doesn't control growth; one-time system-prompt placement (D) loses facts added later.

A customer opens with "This is the third time! Just connect me to a person." The issue (a duplicate newsletter signup) is trivial for the agent. Which action correctly distinguishes the two signals present?

- A** Resolve the trivial issue first, then mention escalation is available.
- B** Run sentiment analysis; escalate only if anger passes a threshold.
- C** Refuse escalation because the agent can handle it, to protect first-contact resolution.

D Honor the explicit human request immediately, regardless of how simple the issue is.

✓ CORRECT



Answer D — An explicit human request is honored immediately. The 'acknowledge-and-offer-resolution' path applies only when the customer is frustrated but has NOT explicitly demanded a human.

Policy documents the agent can read cover refunds and own-site price adjustments but are entirely silent on competitor price matching, which the customer now requests. The lookup tools all succeed. Which is the BEST action?

A Escalate, because policy is silent/ambiguous on this specific request.

✓ CORRECT

B Deny it, since nothing authorizes a competitor match.

C Approve it by analogy to the own-site adjustment rule.

D Ask the customer to prove the competitor's price, then auto-approve.



Answer A — A policy gap/silence on the specific request is an escalation trigger. Improvising approval (B/D) or a hard denial (B) both substitute the agent's judgment for an undefined policy.

Two handoff designs are proposed: (1) the full transcript with every tool result inline, (2) a structured summary with customer ID, root cause, amount, and recommended action. The human reviewer has limited time and no system access. Which is better and what's the failure mode of the other?

A Design 1; more context is always safer for the reviewer — an option that holds only under assumptions this scenario breaks.

B Design 2; the full transcript imposes cognitive overload and buries the decision-relevant facts.

✓ CORRECT

C Design 1; structured summaries omit the customer's exact words.

D Neither; the agent should just hand off the refund amount alone.

 Answer B — A structured handoff gives the human exactly the decision-relevant facts. Raw transcripts overload the reviewer; amount-only omits identifying context and root cause.

Order-lookup results carry 40+ fields each and now dominate context. Three options are on the table. Which BEST reduces tokens while preserving what the agent actually needs, and why are the others worse?

A Move all results to a vector DB and retrieve semantically as needed.

B Convert each result to a prose paragraph the model compresses later.

C Trim each result to the few return-relevant fields before it accumulates.

✓ CORRECT

D Keep all fields but raise max_tokens to fit them.



Answer C — Field trimming keeps the structured data while cutting tokens. A vector DB adds latency/complexity for in-conversation data; prose loses structured precision; raising max_tokens doesn't reduce accumulation.

An aggregation step over 18 sections reliably drops middle-section findings. A teammate proposes a bigger-context model. Why is the ordering fix preferable?

A A bigger context window directly fixes attention to middle content.

B Middle content is unimportant by definition, so it can be dropped.

C Lowering temperature forces uniform attention across positions.

D Position effects persist regardless of window size.

✓ CORRECT



Answer D — 'Lost in the middle' is a position effect, not a capacity limit; a larger window doesn't resolve it. Summaries up front plus section headers do. Front-loading a findings summary and using explicit section headers mitigates them.

After answering verification question 3, the agent re-asks for the name. A junior dev 'fixes' it by adding 'remember prior turns' to the system prompt; it still fails intermittently. What is the actual cause and correct fix?

A History isn't being passed in each request.

✓ CORRECT

B Claude has a 2-turn memory cap.

C The verification tool wipes internal state.

D Temperature is too high; set it to 0 so memory stabilizes.



Answer A — The model is stateless; coherence requires passing history each call. A prompt instruction can't conjure turns that aren't in the request, which is why the 'fix' only intermittently helps. Include the full conversation history per API call.

In 9% of refund flows, the agent processes a refund before identity verification despite a strong prompt mandate. The business needs zero such cases. Which statement is correct?

A Adding few-shot examples of verifying-first will reach zero failures.

B Prompt-based ordering has a non-zero failure rate.

✓ CORRECT

C Routing tools by request type fixes the ordering problem.

D Lowering temperature to 0 makes prompt compliance deterministic.



Answer B — Deterministic compliance for financial operations needs a programmatic gate; prompt/few-shot approaches remain probabilistic and can't guarantee zero failures. Routing addresses availability, not ordering. A programmatic prerequisite gate (block refund until verified ID returned) is required for a hard guarantee.

You have two paused investigations. (X): prior tool results still reflect the current code. (Y): prior results reference files refactored since. What is the correct resumption strategy for each?

A Resume both with `--resume` to retain reasoning.

B Start both fresh to be safe — a quick fix that tends to resurface the same failure later.

C Resume X; for Y start fresh with an injected structured summary of still-valid findings.

✓ CORRECT

D Resume Y; restart X, since X's results are too old to trust.



Answer C — Resume when prior context is mostly valid (X); when prior tool results are stale (Y), a fresh session seeded with a summary is more reliable than resuming on outdated context.

An agent at 55% resolution escalates easy cases and attempts hard policy-exception cases. A PM wants to add (i) sentiment-based auto-escalation and (ii) self-reported confidence routing. Why do both miss the real problem?

- A Both are too expensive to run in production.
- B Sentiment works, but confidence routing must use a 1–100 scale.
- C Both work; they should be combined for redundancy.
- D Neither correlates with actual case complexity (the real issue).**

✓ CORRECT



Answer D — The failure is unclear decision boundaries on complexity. Sentiment doesn't track complexity, and self-reported confidence is poorly calibrated (the agent is confidently wrong on hard cases). Explicit criteria + examples fix the boundary. Explicit escalation criteria with few-shot examples address it directly.

SECTION



Batch API & Extraction

ADVANCED — SLA math, semantic validation, retry limits

Q13-Q20

You must guarantee results within 30 hours; batch jobs can take up to 24 hours each with no SLA. A teammate suggests a 24-hour submission cadence 'since jobs usually finish in 6 hours'. Why is that unsafe, and what cadence works?

A Worst case a 24h-cadence job submitted just before a slow window finishes at ~48h.

✓ CORRECT

B It's fine; 6-hour typical completion keeps you well under 30 hours.

C Switch to synchronous; batch can never meet a 30-hour SLA.

D Submit once at hour 0 and poll — a workaround that adds complexity without improving reliability.



Answer A — SLA math must use the worst case (24h window), not the typical 6h. A 24h cadence risks ~48h total; a ~4–6h cadence bounds worst-case completion under 30h. Submit on a ~4–6h cadence so worst-case stays under 30h.

An engineer wants to move a multi-turn tool-calling support agent to the batch API for the 50% discount. Which is the single disqualifying reason?

A Batch responses can't be correlated to requests.

B The batch API can't execute tools mid-request and feed results back.

✓ CORRECT

C Batch mode doesn't allow system prompts.

D Batch mode forbids JSON schemas — a route that optimizes effort over correctness here.



Answer B — The batch API has no multi-turn tool-calling within a request; a tool-using agent loop can't run there. (custom_id does allow correlation, so A is false.) No multi-turn tool calling in one request.

A 500-doc batch returns with 11 failures: 7 exceeded context limits, 4 hit transient service errors. What is the most efficient, correct recovery?

- A Resubmit all 500 to be safe — a plausible-sounding move that misreads the root cause.
- B Drop all 11 and ship the 489 successes — a change that leaves the real failure mode unaddressed.
- C Resubmit only the 11 by custom_id: chunk the 7 oversized docs, re-send the 4 transient ones as-is.**
- D Chunk all 500 documents and resubmit — an option that scales poorly as the system grows.

✓ CORRECT



Answer C — Use custom_id to resubmit only failures, applying the right fix per cause: chunk the oversized ones, simply retry the transient ones. Re-running everything wastes cost; chunking all 500 is unnecessary.

On validation failure you retry with the error appended. Field A (date) fails because the date is absent from the source; field B fails because the model emitted MM-DD-YYYY instead of ISO. Which retries will succeed?

A Both A and B will succeed with enough retries.

B A will succeed (the model can infer dates); B will not.

C Neither; retries never help once validation fails.

D B will succeed (format error).

✓ CORRECT



Answer D — Retry-with-feedback fixes format/structural errors (D). It cannot supply information that doesn't exist in the source (A); recognizing absence vs malformation is the key skill. A will not (information absent from source).

Using `tool_use` with a strict schema, syntax errors vanish, but some invoices have line items that don't sum to the stated total and a tax value placed in the subtotal field. What's the accurate conclusion?

A Strict schemas remove syntax errors but not semantic errors.

✓ CORRECT

B The schema is broken; regenerate it.

C Set `tool_choice`: "any" to enforce arithmetic and field placement.

D These are syntax errors a stricter schema will eliminate.



Answer A — Tool-use schemas guarantee well-formed JSON, not semantic correctness. Wrong-field placement and non-summing items are semantic; detect them with explicit self-check fields. Add self-check fields (`calculated_total` vs `stated_total`, `conflict_detected`) to catch them.

Required field `tax_id` is fabricated when absent. A dev makes it nullable, but ALSO adds 'always provide a `tax_id`' to the prompt. Fabrication persists. What's wrong?

A Nullable fields don't work with `tool_use`.

B The prompt instruction re-imposes a de-facto requirement.

✓ CORRECT

C The schema needs a regex pattern instead of nullable.

D Temperature must be 0 for null to be returned.



Answer B — Making the field nullable is correct, but a prompt that says 'always provide' contradicts it and pressures fabrication. Align the prompt with the nullable design. Remove it so the model can legitimately return null.

An extractor nails table-format invoices but returns nulls on prose-format ones; instructions alone haven't helped. Which addition most directly fixes the null-on-prose problem?

A Make every field required to force extraction — an easy-to-reach answer that doesn't survive closer scrutiny.

B Increase max_tokens so the whole doc is read — a tactic that trades long-term reliability for short-term ease.

C Add few-shot examples demonstrating correct extraction from varied structures, including prose, to generalize beyond pre-specified formats.

✓ CORRECT

D Reject prose invoices as unsupported — a step that appears safe but reduces observability.



Answer C — Few-shot examples across varied document structures address empty/null extraction and let the model generalize to novel layouts, which instructions alone don't.

A pipeline reports 97% overall accuracy and leadership wants to drop human review entirely. Which single check most protects against a hidden failure mode?

A

Confirm latency is acceptable.

B

Re-run the whole set through the batch API to confirm 97%.

C

Increase the sample size of low-confidence reviews only.

D

Analyze accuracy by document type and field segment.

✓ CORRECT



Answer D — Aggregates can hide weak segments. Validate accuracy by document type and field before reducing review; route remaining low-confidence/ambiguous cases to humans. A 97% aggregate can mask poor performance on a specific type or field.

SECTION



Tool Design

ADVANCED — selection, distribution, enforcement, error contracts

Q21-Q34

search_records and find_entries have one-line descriptions and overlapping inputs; the model misroutes. Two senior engineers disagree: one wants 8 few-shot routing examples, the other wants richer descriptions. As a FIRST step, which is better and why?

A Richer descriptions (inputs, examples, boundaries, when-to-use-vs-other).

✓ CORRECT

B Few-shot examples; they directly demonstrate routing.

C A keyword pre-router; it removes model ambiguity entirely.

D Consolidate the tools; one tool can't be misrouted — a fix that addresses configuration when the issue is design.



Answer A — Descriptions are the primary signal for selection; fixing minimal descriptions is the high-leverage first step. Few-shot adds token overhead without fixing the root cause; a router over-engineers; consolidation is a heavier architectural change.

A synthesis subagent with 18 tools sometimes runs web searches it shouldn't. A dev proposes detailed descriptions for all 18 to stop misuse. Why is scoping the tool set the better lever here?

A Descriptions can't be written for 18 tools.

B Selection reliability degrades with tool count.

✓ CORRECT

C tool_choice: "any" would prevent all misuse.

D Web search tools are inherently unsafe and must be deleted globally.



Answer B — Beyond a point, more tools (even well-described) increase decision complexity and misuse. Scoping each agent to role-relevant tools is the structural fix; a scoped cross-role tool covers high-frequency needs. Reducing to the role-relevant 4–5 (plus limited cross-role tools) attacks the decision-complexity root cause that descriptions alone won't.

Synthesis needs frequent simple fact-checks (85%) plus occasional deep verification (15%), today all routed through the coordinator (+40% latency). Compare giving it (A) all web tools vs (B) a scoped verify_fact tool with complex cases still routed. Which and why?

A All web tools, so it never round-trips — a choice that adds effort without changing the outcome.

B Batch all verifications to the end of the pass — an approach that the production logs would later contradict.

C A scoped verify_fact tool for the common simple case; route complex verification through the coordinator.

✓ CORRECT

D Pre-cache extra context for every source up front.



Answer C — A scoped cross-role tool serves the 85% case while complex cases keep separation of concerns. All web tools over-provision; batching creates blocking dependencies; speculative caching can't predict needs. Least privilege without over-provisioning.

Your tools return only 'Operation failed'. You want the agent to retry transient issues, fix bad inputs, explain policy denials, and stop on permission errors. Which response contract enables ALL four behaviors?

- A An HTTP status code only — a move that confuses tool availability with tool ordering.
- B A longer prose message with no fields — a path that pushes the cost onto a later step instead.
- C A stack trace for the agent to parse — an option that relies on behavior the model can't guarantee.

D **errorCategory (transient/validation/business/permission) + isRetryable + a human-readable description (+ retrieable:false with a customer-friendly note for business rules).**

✓ CORRECT



Answer D — Category + retryable flag + readable description (and customer-friendly business-rule notes) lets the agent choose retry vs fix vs explain vs stop. A bare status or prose can't drive all four decisions reliably.

inventory_lookup returns isError both when the DB times out AND when no SKU matches. The agent now retries 'no matches' forever and sometimes reports outages as 'not found'. What's the precise fix?

A Distinguish access failures (timeout → retryable error) from valid empty results (no matches → successful response with empty set).

✓ CORRECT

B Always escalate any isError to a human — a change that masks the failure rather than resolving it.

C Treat both as success to stop the retries — a tactic that works on the happy path but not the edge cases.

D Treat both as fatal and terminate the workflow — an answer that sounds rigorous yet ignores the stated constraint.



Answer A — A no-match is a successful query with an empty result, not an error; a timeout is an access failure needing a retry decision. Conflating them causes both pathologies described.

A subagent hits a transient timeout, recovers on its own retry, then later hits a hard permission error it can't resolve. What should it send the coordinator in each case?

A Propagate both immediately and stop — a choice that improves one metric while degrading another.

B Resolve the transient one locally (no propagation).

✓ CORRECT

C Suppress both and return empty success.

D Terminate the workflow on the first timeout.



Answer B — Subagents recover locally from transient failures and propagate only unresolved errors, including attempted actions and partial results, so the coordinator can decide intelligently. Propagate only the permission error, with what was attempted and any partial results.

An overloaded analyze tool (extract/summarize/verify) behaves unpredictably. Compare (A) adding a 'mode' string argument vs (B) splitting into purpose-specific tools. Which better fixes selection reliability and why?

A Mode argument; one tool is simpler for the model to find — an approach that treats the symptom rather than the cause.

B Keep one tool and force it with tool_choice each turn — a tempting shortcut that skips the underlying fix.

C Split into `extract_data_points`, `summarize_content`, `verify_claim_against_source` with distinct descriptions and I/O contracts.

✓ CORRECT

D Remove the tool and do everything in prose — a step that introduces a new dependency for little gain.



Answer C — Purpose-specific tools with clear descriptions and I/O contracts remove the ambiguity. A mode argument keeps one fuzzy description and shifts the ambiguity into a parameter the model still mis-sets. The within-tool ambiguity.

A category enum can't cover future values and the model both force-fits unknowns AND mislabels genuinely ambiguous cases. Which schema design handles BOTH cleanly?

A Drop the enum; accept any free-text string — a choice that looks simpler but shifts the problem elsewhere.

B Make the field required and non-nullable to force a decision.

C Pre-enumerate 300 categories so 'other' is never needed.

D Add an "other" value with a detail string for novel cases, plus an "unclear" value for ambiguity.

✓ CORRECT



Answer D — 'other' + detail handles novel categories without fabrication; 'unclear' handles ambiguity without forcing a wrong label. Free-text loses structure; non-nullable forces mislabeling; mega-enums are unmaintainable.

extract_metadata must run before any enrichment tool, every time. Which configuration guarantees ordering, and what's the correct follow-up?

A Force { "type": "tool", "name": "extract_metadata" } on the first turn, then process enrichment in subsequent turns.

✓ CORRECT

B tool_choice: "auto" with extract_metadata listed first in the array.

C tool_choice: "any" so the model always calls a tool — an option that holds only under assumptions this scenario breaks.

D Put 'call metadata first' in every other tool's description.



Answer A — Forced tool selection guarantees the specific first call; remaining steps proceed in follow-up turns. 'auto' only permits ordering; 'any' guarantees some tool, not which.

Tool descriptions are well-differentiated, yet the model over-selects `summarize_content`. Logs show the system prompt repeats 'always summarize the situation'. Few-shot examples don't fix it. What's the correct diagnosis and action?

- A Add more few-shot examples to outweigh the bias.
- B Keyword-sensitive system-prompt wording is creating an unintended tool association.**
- C Raise temperature so selection varies — a quick fix that tends to resurface the same failure later.
- D Reorder the tools array to deprioritize `summarize_content`.

✓ CORRECT



Answer B — System-prompt keywords can override even good descriptions. The fix is to revise the keyword-heavy wording, not pile on examples or shuffle ordering. Revise the prompt wording.

A document-analysis agent has a generic `fetch_url` and occasionally pulls arbitrary/unsafe pages. Which redesign best constrains it without removing the needed capability?

A Keep `fetch_url`; add 'be careful' to the prompt.

B Give the agent `fetch_url` plus more tools for options.

C Replace `fetch_url` with `load_document` that validates the URL is an allowed document source.

✓ CORRECT

D Force `tool_choice`: "any" so it always acts — a workaround that adds complexity without improving reliability.



Answer C — Replacing a generic tool with a constrained alternative (`load_document` validating document URLs) limits misuse while preserving capability — the canonical pattern.

A capable custom MCP search tool is consistently passed over for built-in Grep. Disabling Grep is off the table. What most directly shifts adoption to the MCP tool?

A Rename it to sort ahead of Grep — a plausible-sounding move that misreads the root cause.

B Force it via `tool_choice` on every turn — an option that scales poorly as the system grows.

C Lower the model temperature — a route that optimizes effort over correctness here.

D Enhance the MCP tool's description to explain its capabilities and outputs in detail vs the built-in.

✓ CORRECT



Answer D — Detailed MCP descriptions prevent the agent from defaulting to built-ins like Grep over more capable MCP tools; naming tricks and forcing are blunt instruments.

Developers keep dismissing one class of review findings but you can't see which code constructs trigger them. Which instrumentation enables systematic false-positive analysis?

A Add a `detected_pattern` field recording the construct that triggered each finding.

✓ CORRECT

B Add a free-text notes field — a change that leaves the real failure mode unaddressed.

C Add a timestamp to each finding.

D Attach the full file to each finding.

 Answer A — A `detected_pattern` field lets you analyze which constructs drive dismissed findings, so prompts for high-false-positive categories can be tuned (or temporarily disabled to restore trust).

Your review prompt has many pre-specified 'acceptable pattern' rules, yet novel-but-acceptable patterns still get flagged. Which approach best reduces false positives WHILE generalizing to unseen patterns?

A

Keep adding explicit acceptable-pattern rules for each new case — a tactic that trades long-term reliability for short-term ease.

B

Provide a few targeted few-shot examples distinguishing acceptable patterns from genuine issues.

✓ CORRECT

C

Instruct the model to 'be conservative' and 'only high-confidence findings'.

D

Disable the category permanently — a step that appears safe but reduces observability.



Answer B — Few-shot examples that contrast acceptable patterns with real issues reduce false positives and generalize to novel cases. Endless explicit rules don't generalize; vague 'be conservative' instructions don't improve precision. Generalization.

SECTION



MCP Design Patterns

ADVANCED — scoping, resources, hooks vs prompts, propagation

Q35-Q46

A shared GitHub MCP server must reach every teammate via the repo and authenticate with a token that must NOT be committed; a personal experimental server is yours alone. Which configuration satisfies all three constraints?

A Both servers in ~/.claude.json with inline tokens.

B Both in project .mcp.json with the literal token, gitignored.

C Shared server in project .mcp.json using \${GITHUB_TOKEN} env expansion.

✓ CORRECT

D Shared server in CLAUDE.md — an easy-to-reach answer that doesn't survive closer scrutiny.



Answer C — Project .mcp.json (committed) shares the server; \${GITHUB_TOKEN} expansion keeps the secret out of version control; user ~/.claude.json scopes the experimental server to you. Personal server in user ~/.claude.json.

Agents waste many tool calls discovering what issues, docs, and schemas exist before acting. A dev suggests adding more discovery tools. Why are MCP resources the better answer?

- A More tools are faster than resources for discovery.
- B Resources execute actions more efficiently than tools.
- C Resources increase the context window — a fix that addresses configuration when the issue is design.

D Resources expose content catalogs so agents see available data without exploratory tool calls.

✓ CORRECT



Answer D — MCP resources surface content catalogs (issues, doc hierarchies, schemas), cutting exploratory calls. Adding more action tools increases decision complexity instead of reducing discovery cost. Tools are for actions, resources for catalogs.

A rule states no single action may transfer over \$1,000. The prompt enforces it but ~3% slip through. Leadership needs a hard guarantee. Which control type and placement is correct?

A A tool-call interception hook that blocks transfers over \$1,000 and redirects to escalation.

✓ CORRECT

B Strengthen the prompt and also add the rule to CLAUDE.md.

C A PostToolUse hook that flags transfers after they execute.

D Lower temperature to 0 to make the prompt deterministic — a choice that adds effort without changing the outcome.



Answer A — A guaranteed business rule needs an interception hook that blocks the action before it executes. A PostToolUse hook fires after the fact (too late); prompts/temperature remain probabilistic. Deterministic enforcement.

Three MCP tools return timestamps as Unix epoch, ISO 8601, and numeric status codes; the agent reasons inconsistently across them. Which hook, and why not the alternatives?

- A A PreToolUse hook rewriting the request before the call.
- B A PostToolUse hook that normalizes the heterogeneous results before the model processes them.**
- C A system-prompt instruction to mentally convert formats.
- D tool_choice: "any" to standardize outputs — an approach that the production logs would later contradict.

✓ CORRECT



Answer B — PostToolUse intercepts tool RESULTS for transformation before the model reads them. PreToolUse acts on the outgoing call (wrong direction); prompt conversion is unreliable; tool_choice doesn't transform data.

An MCP tool fails and you want the agent to make a smart recovery decision rather than blindly retry. What's the most complete signaling approach?

- A Return success with empty content and let the agent infer failure.
- B Drop the network connection so the agent notices.
- C Set the `isError` flag AND include structured metadata (category, retryable, what was attempted).**
- D Return a 200 with the word 'error' in prose — a move that confuses tool availability with tool ordering.

✓ CORRECT



Answer C — `isError` communicates failure; structured metadata (category/retryable/attempt context) enables intelligent recovery instead of blind retries. Empty-success hides failure; dropping connections is opaque.

You need a standard Jira integration and a bespoke internal-deployment workflow. Effort is limited. Which allocation is recommended and what's the rationale?

A

Build both custom for full control.

B

Use community servers for both, accepting a poor fit for the internal workflow.

C

Avoid MCP; call REST APIs directly from prompts.

D

Use a community MCP server for Jira (standard).

✓ CORRECT



Answer D — Prefer community servers for standard integrations like Jira; reserve scarce custom-build effort for genuinely team-specific workflows that no community server covers. Reserve a custom server for the internal-specific workflow.

A teammate claims that with three configured MCP servers you must enable one per session. What is accurate, and what does this imply for tool-set size?

A Incorrect; all configured servers' tools are discovered at connection time and available simultaneously.

✓ CORRECT

B Correct; only one server's tools load per session.

C Only the most recently added server loads.

D Tools must be re-imported in each prompt — an option that relies on behavior the model can't guarantee.



Answer A — All configured servers' tools are available at once; this is why uncontrolled server/tool growth can degrade selection reliability and why scoping matters. So total tool count (and selection complexity) can grow quickly.

Agents must frequently READ your database schema but never MODIFY it. Should it be exposed as a tool or a resource, and why does the distinction matter?

A A tool, because anything an agent consumes is a tool.

B A resource — a read-only content catalog, not an action.

✓ CORRECT

C Neither; paste the schema into every prompt.

D A tool forced via tool_choice each turn.



Answer B — Read-only catalogs (schemas) are resources; tools are for actions. Resource modeling also cuts the exploratory calls agents would otherwise make to discover the schema. Modeling it as a resource also reduces exploratory tool calls.

An agent occasionally calls a high-impact MCP write tool before a required verification tool completes. Which control guarantees ordering, and why not the alternatives?

A

A system-prompt instruction mandating verification first.

B

Few-shot examples of verifying first.

C

A programmatic prerequisite that blocks the write tool until verification returns.

✓ CORRECT

D

A request-type routing classifier — a path that pushes the cost onto a later step instead.



Answer C — Critical ordering needs a programmatic prerequisite gate (deterministic). Prompts/few-shot remain probabilistic; routing addresses which tools are available, not the order they're called.

One source the synthesis agent needed was unavailable, so part of the topic is unsupported. Which output design is correct, and why are the others anti-patterns?

- A** Silently omit the unsupported area for a cleaner report.
- B** Fabricate plausible findings to fill the gap — a tactic that works on the happy path but not the edge cases.
- C** Fail the entire synthesis — a change that masks the failure rather than resolving it.

D Annotate coverage: mark well-supported findings vs areas with gaps due to unavailable sources.

✓ CORRECT



Answer D — Coverage annotations preserve honesty about gaps. Silent omission hides them, fabrication is harmful, and failing the whole synthesis on one missing source is the terminate-on-single-failure anti-pattern.

A web-search subagent times out on a complex query. Which propagation design BEST enables coordinator recovery, and what's wrong with a generic 'search unavailable'?

A Return structured error context: failure type, attempted query, partial results, and alternative approaches.

✓ CORRECT

B Return 'search unavailable' after exhausting internal retries — an answer that sounds rigorous yet ignores the stated constraint.

C Catch the timeout and return an empty result marked successful.

D Propagate the exception to a top-level handler that ends the workflow.



Answer A — Structured error context lets the coordinator retry differently, try alternatives, or proceed with partials. Generic statuses hide context; empty-success suppresses the error; top-level termination kills recoverable runs. Generic statuses hide the context needed to recover.

Why is `${TOKEN}` environment-variable expansion in `.mcp.json` preferred over inlining the secret and gitignoring the file?

A Inlining is fine; gitignore fully protects secrets — a tempting shortcut that skips the underlying fix.

B Env expansion keeps the secret out of the committed file entirely, so the shared config can be version-controlled without leaking credentials.

✓ CORRECT

C Env expansion makes the server faster — a choice that improves one metric while degrading another.

D Env expansion is required for the server to start — a step that introduces a new dependency for little gain.



Answer B — Env-var expansion lets the shared `.mcp.json` be committed and used by the team without ever containing the secret value; gitignoring an inlined secret risks accidental commits and can't be shared safely.

SECTION



Agentic Systems & Code Agents

ADVANCED — decomposition, context, recovery, mode selection

Q47-Q52

Research on 'the future of work' yields a report covering only 'remote work', 'hybrid offices', and 'coworking spaces'. Each subagent succeeded at its assigned task; logs show the coordinator chose exactly those three subtopics. A dev blames the synthesis agent's gap-detection. Who's right?

A The synthesis agent — it should have detected the missing domains — an option that holds only under assumptions this scenario breaks.

B The web-search agent — its queries were malformed — a choice that looks simpler but shifts the problem elsewhere.

C The coordinator — overly narrow decomposition omitted whole relevant domains (automation, gig economy, reskilling, etc.).

✓ CORRECT

D The document-analysis agent — an approach that treats the symptom rather than the cause.



Answer C — When subagents execute correctly but whole domains are missing, the root cause is narrow decomposition by the coordinator. Downstream agents only covered what they were assigned.

Your coordinator delegates to three independent subagents but issues each Task call in a separate turn, so they run sequentially. allowedTools already includes Task. What's the fix and the expected benefit?

- A** Add Task to allowedTools again — a quick fix that tends to resurface the same failure later.
- B** Give each subagent tool_choice: "any" — a workaround that adds complexity without improving reliability.
- C** Resume each subagent with --resume to overlap them.

D Emit multiple Task calls in a single coordinator response to run them in parallel, reducing total latency.

✓ CORRECT



Answer D — Parallel subagents are spawned by emitting multiple Task calls in one response. (Task is already allowed, so A is irrelevant.) This cuts wall-clock latency vs sequential turns.

A synthesis subagent produces thin, poorly-cited output. The coordinator assumed context inheritance and passed only a topic string. Which fix addresses BOTH the thinness and the lost attribution?

A

Pass the complete prior findings in the prompt as structured claim-source mappings (claim, excerpt, source, date) the synthesis must preserve.

✓ CORRECT

B

Retry; subagents inherit parent context automatically — a plausible-sounding move that misreads the root cause.

C

Give synthesis the web tools to refetch everything itself — an option that scales poorly as the system grows.

D

Lower the synthesis temperature — a route that optimizes effort over correctness here.



Answer A — Subagents don't inherit context; pass findings explicitly. Using structured claim-source mappings simultaneously fixes thinness (full findings) and provenance (attribution preserved through synthesis).

Deep into a large-codebase session, the agent starts citing 'typical patterns' instead of the specific classes it found earlier and gives inconsistent answers. Which combination best counteracts this?

A Raise temperature to encourage richer answers.

B Maintain scratchpad files of key findings and reference them.

✓ CORRECT

C Read all files into context at once to ground the model.

D Restart the session from scratch on every drift.



Answer B — Drifting to 'typical patterns' signals context degradation. Scratchpads persist findings; /compact and Explore-subagent isolation keep the main context lean. Reading everything worsens it; restarting loses progress. Use /compact and Explore-subagent delegation to isolate verbose discovery.

A multi-agent run must survive a mid-run crash and resume cleanly without re-exploring everything. Which design is correct?

A Rely on the model recalling state after restart.

B Keep all state only in the context window.

C Each agent exports structured state to a known location.

✓ CORRECT

D Re-run the whole pipeline from zero with no persisted state.



Answer C — Structured state exports plus a coordinator-loaded manifest on resume enable crash recovery and targeted continuation, avoiding full re-exploration. On resume the coordinator loads a manifest and injects it into agent prompts.

Task: migrate a logging library used across 45+ files (multiple viable approaches, differing infra), THEN apply the chosen approach. What's the optimal mode strategy?

- A Direct execution throughout — a change that leaves the real failure mode unaddressed.
- B Plan mode for the whole task, including every file edit.
- C Direct execution with one large upfront instruction — a step that appears safe but reduces observability.
- D Plan mode to explore and choose an approach, then direct execution to implement the chosen plan.**

✓ CORRECT



Answer D — Architecturally significant, multi-approach work warrants plan mode for investigation/design; implementation of the chosen plan is then well-scoped for direct execution. Complexity is already known, so 'switch later' (A) is wrong.

SECTION



AI Fluency Framework

ADVANCED — competency boundaries, loops, assessment design

Q53-Q66

A user decides a literature review is suitable for AI and then writes a detailed prompt specifying scope, sources to prioritize, and output format. Which competencies map to the two acts, in order?

A Delegation (deciding it's suitable and how to divide work), then Description (communicating the task clearly).

✓ CORRECT

B Description, then Delegation — an easy-to-reach answer that doesn't survive closer scrutiny.

C Discernment, then Diligence — a tactic that trades long-term reliability for short-term ease.

D Delegation, then Discernment — a fix that addresses configuration when the issue is design.



Answer A — Deciding what to entrust to AI and how to split the work is Delegation; crafting the clear, contextual prompt is Description.

A learner (1) notices the AI invented a citation and removes it, then (2) adds a disclosure note that AI assisted the document and takes responsibility for the final claims. Map (1) and (2).

A (1) Diligence, (2) Discernment — a choice that adds effort without changing the outcome.

B (1) **Discernment — critical evaluation.**

✓ CORRECT

C (1) Description, (2) Delegation

D (1) Delegation, (2) Discernment



Answer B — Catching the fabricated citation is Discernment (critical evaluation); disclosing AI use and owning the outcome is Diligence (responsible, transparent, accountable use). (2) Diligence — transparency and accountability.

An analyst prompts, evaluates the output, refines the prompt based on flaws found, evaluates again, iterating to a strong result. This iterative loop is best characterized as the interplay of which two competencies?

A

Delegation and Diligence

B

Discernment and Diligence

C

Description and Discernment

✓ CORRECT

D

Delegation and Description



Answer C — Crafting/refining prompts (Description) and critically evaluating outputs (Discernment) together form the core refinement loop.

Choosing NOT to use AI for a sensitive legal filing, and documenting that decision and its reasoning for the record, most directly engages which pairing?

A Description and Discernment

B Discernment and Description

C Delegation and Description

D Delegation and Diligence

✓ CORRECT



Answer D — The decision about whether/what to delegate is Delegation; transparent, accountable documentation of that responsible choice is Diligence.

An AI returns a fluent, authoritative answer that happens to be wrong. A student accepts it because it 'sounded confident'. Which competency did the student most fail to exercise?

A Discernment

✓ CORRECT

B Delegation

C Description

D Diligence



Answer A — Failing to critically evaluate a fluent-but-wrong answer is a Discernment failure; fluency is not correctness.

An instructor grades ONLY the final AI-assisted deliverable against a rubric, ignoring how the student worked with the AI. Name the type and its key blind spot.

A Process-based; blind to the final quality.

B Outcome-based; blind to HOW the AI was used (delegation, prompting, verification).

✓ CORRECT

C Reflection-based; blind to the rubric — an approach that the production logs would later contradict.

D Peer-based; blind to the instructor's view.



Answer B — Outcome-based assessment judges the product and can miss the collaboration process, which process- and reflection-based methods capture.

An instructor examines a student's delegation choices, prompt iterations, and verification steps to grade collaboration skill. Name the type and what it captures that outcome-only grading misses.

A Outcome-based; captures the final artifact.

B Reflection-based; captures the student's narrative only.

C Process-based; captures HOW the learner collaborates with AI (decisions, iterations, checks).

✓ CORRECT

D Automated; captures nothing human — a move that confuses tool availability with tool ordering.



Answer C — Process-based assessment evaluates the working method with AI — delegation, prompting iterations, and verification — beyond the final product.

Students submit a narrative explaining WHY they delegated certain parts, HOW they verified outputs, and what they'd change. Name the type and the competency it most surfaces.

A Outcome-based; surfaces product quality — a path that pushes the cost onto a later step instead.

B Process-based only; surfaces nothing about reasoning — a change that masks the failure rather than resolving it.

C Rubric-only; surfaces formatting — an option that relies on behavior the model can't guarantee.

D Reflection-based; surfaces metacognition about their AI collaboration (and Diligence/Discernment reasoning).

✓ CORRECT



Answer D — Reflection-based assessment elicits the learner's reasoning and self-evaluation of their AI collaboration, surfacing metacognition the artifact alone doesn't show.

A program wants assessment reflecting BOTH deliverable quality AND collaboration skill, while resisting gaming. Which design is strongest?

A Combine outcome, process, and reflection methods so product quality and collaboration are both captured.

✓ CORRECT

B Outcome-only, for objectivity — a tactic that works on the happy path but not the edge cases.

C Reflection-only, to save grading time — a choice that improves one metric while degrading another.

D Ban AI so assessment stays simple — an answer that sounds rigorous yet ignores the stated constraint.



Answer A — Blending outcome, process, and reflection captures both the product and how effectively the learner worked with AI, and is harder to game than any single method.

An educator replaces a definition quiz with open-ended scenarios. What is the strongest pedagogical rationale grounded in the framework?

A Scenarios are cheaper to grade than quizzes — a step that introduces a new dependency for little gain.

B Scenarios require applying the 4Ds together with contextual judgment, not just recalling terms.

✓ CORRECT

C Scenarios remove the need for human evaluation.

D Scenarios force every learner to the same answer.



Answer B — Scenario-based learning makes learners exercise Delegation, Description, Discernment, and Diligence together in realistic context, beyond rote recall.

Faculty debate teaching AI fluency as one standalone module vs integrating it across disciplinary courses. Which argument best supports integration?

A Integration removes the need to teach the discipline — a choice that looks simpler but shifts the problem elsewhere.

B Integration eliminates the need for assessment — a tempting shortcut that skips the underlying fix.

C Embedding fluency in authentic disciplinary tasks builds transferable, contextual judgment a standalone module struggles to develop.

✓ CORRECT

D Integration guarantees students never misuse AI — an approach that treats the symptom rather than the cause.



Answer C — Integration into real disciplinary work develops transferable, contextual judgment; a single isolated module is less able to achieve that transfer.

Two prompts for the same task: (P1) 'Summarize this.' (P2) 'Summarize this 12-page report for a non-technical exec in <150 words, lead with the recommendation, then 3 supporting points.' Which competency does P2 exemplify, and why is it stronger?

- A** Discernment; it evaluates the output.
- B** Delegation; it decides to use AI.
- C** Diligence; it discloses AI use — an option that holds only under assumptions this scenario breaks.

D Description; it supplies audience, length, structure, and intent so the AI can do its part well.

✓ CORRECT



Answer D — P2 demonstrates strong Description: clear context, constraints, and structure that communicate the task precisely.

Effective Delegation in the framework rests on awareness of several things before handing work to AI. Which set best reflects that awareness?

A The problem, the process, and the platform's capabilities.

✓ CORRECT

B Only the model's price and speed.

C Only the output format desired — a quick fix that tends to resurface the same failure later.

D Only whether disclosure is required.



Answer A — Delegation involves awareness of the problem, the process for solving it, and the platform's (AI's) capabilities, informing what to delegate and how to divide the work. To decide what to delegate and how.

A practitioner: decides AI suits a task (i), writes a precise prompt (ii), spots and fixes an error in the output (iii), and discloses AI assistance while owning the result (iv). Match i–iv to the 4Ds in order.

A Description, Delegation, Diligence, Discernment

B Delegation, Description, Discernment, Diligence

✓ CORRECT

C Delegation, Discernment, Description, Diligence

D Discernment, Description, Delegation, Diligence

 Answer B — (i) Delegation, (ii) Description, (iii) Discernment, (iv) Diligence — the four competencies in their natural working order.

W R A P - U P

Summary



Context & Escalation 18%

Stateless model, case-facts blocks, lost-in-the-middle, and when to escalate.



Batch API & Extraction 12%

50% cost vs 24h window, custom_id recovery, nullable schemas, retry limits.



Tool Design 21%

Descriptions drive selection; scope per role; typed errors; forced/any tool_choice.



MCP Design Patterns 18%

Project vs user scope, env-var secrets, resources vs tools, hooks for guarantees.



Agentic Systems 12%

Coordinator decomposition, parallel Task spawns, explicit context, crash recovery.



AI Fluency 19%

The 4Ds — Delegation, Description, Discernment, Diligence — and assessment types.

REMEMBER THIS

Key Takeaways



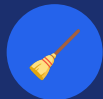
Hard rules need hard enforcement

When a question says "guarantee," "must," or "zero failures," choose programmatic hooks/prerequisite gates — not prompt wording.



Structure your handoffs & errors

Typed error metadata (errorCategory, isRetryable) and compact escalation summaries beat raw transcripts and generic failures.



Practice context hygiene

Trim verbose tool outputs, keep a persistent case-facts block, and counter lost-in-the-middle with summaries + section headers.



Escalate on the right triggers

Explicit human request, policy gap/silence, or no meaningful progress — not sentiment or self-reported confidence.



Tool selection lives in descriptions

Differentiate similar tools, scope each agent to its role, and prefer the proportionate first fix over new ML infrastructure.



Match the API to the latency

Batch API for overnight/latency-tolerant jobs; synchronous API for blocking, interactive, or multi-turn tool-calling work.

BEFORE YOU SIT

Exam Tips

- 1 Read the stem's exact ask** — Distinguish "most effective FIRST step" from "BEST/MOST effective" — the proportionate fix often wins over the maximal one.
- 2 Prefer determinism on "must"** — If a rule must hold every time (money, identity, ordering), pick hooks/gates over prompt instructions, which fail probabilistically.
- 3 Find the real root cause** — In multi-agent failures, blame the agent that made the decision (often the coordinator), not downstream agents working within their scope.
- 4 Reject over-engineering** — When prompt/description tweaks haven't been tried, a new classifier or ML pipeline is usually the wrong, heavier answer.
- 5 Classify errors precisely** — Separate transient vs validation vs business vs permission, and access failures vs valid empty results, before choosing recovery.
- 6 Let stop_reason drive loops** — Continue on "tool_use", stop on "end_turn" — never terminate on text phrases or arbitrary iteration caps.
- 7 Ignore answer length & letter** — Correct options here are randomized in position and length — decide on the concept, not the longest or most-qualified-sounding choice.